

Relazione SkyTracks

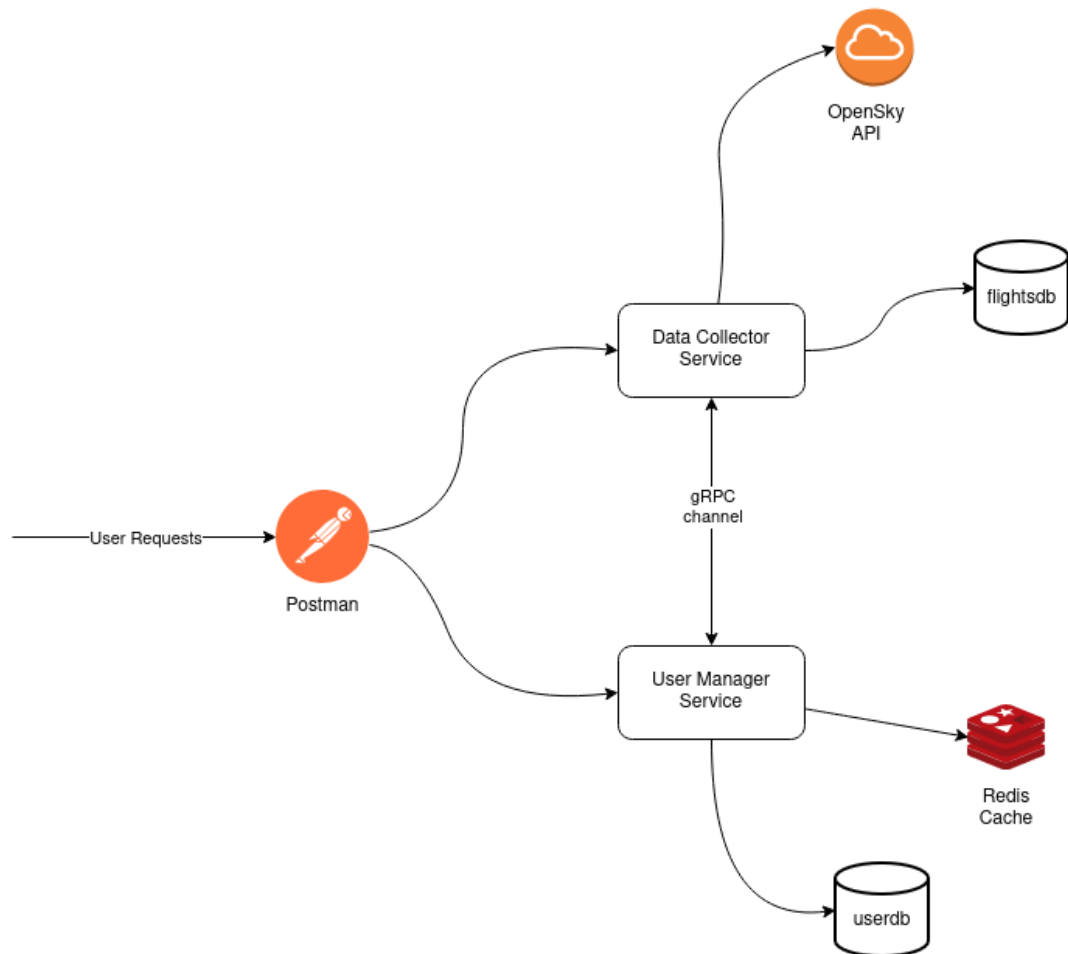
Homework 1 – DSBD

Fabrizio Fichera

1. Descrizione Progetto

Il progetto consiste nello sviluppo di un sistema distribuito basato su microservizi, containerizzato tramite Docker, finalizzato alla gestione degli utenti e al recupero di dati sui voli aerei da OpenSky Network. Il sistema è composto da due servizi principali: lo **User Manager**, responsabile della registrazione e della cancellazione degli utenti secondo una politica *at-most-once*, e il **Data Collector**, che gestisce la lista degli aeroporti di interesse degli utenti, recupera periodicamente le informazioni sui voli e calcola statistiche, come la media del numero di voli negli ultimi N giorni. Il sistema include inoltre componenti essenziali quali i database, utilizzati per memorizzare i dati degli utenti e dei voli, e un servizio di cache impiegato per supportare l'implementazione della politica di idempotenza. La comunicazione tra i servizi avviene tramite gRPC, garantendo efficienza e robustezza, mentre Docker Compose ne consente l'orchestrazione, assicurando modularità, scalabilità e facilità di manutenzione.

2.Schema Architeturale



Componenti del sistema:

- **Client (Postman)**

Utilizzato per simulare le richieste degli utenti tramite interfacce REST. Le richieste possono essere indirizzate sia al servizio User Manager sia al servizio Data Collector, in base alla funzionalità richiesta.

- **User Manager Service**

Responsabile della gestione degli utenti, incluse le operazioni di registrazione e cancellazione.

- Interagisce con un database dedicato (**userdb**) per la persistenza delle informazioni. Esso contiene la tabella users usata per gestire le informazioni degli utenti.
- Utilizza una **cache Redis** per supportare l'implementazione della politica di idempotenza, necessaria a garantire il comportamento *at-most-once* nelle operazioni di registrazione.
- **Data Collector Service**

Gestisce le richieste e le informazioni relative ai voli e aeroporti.

 - Recupera periodicamente le informazioni sui voli in partenza e in arrivo interrogando le **API di OpenSky Network**.
 - Memorizza i dati ottenuti in un database dedicato (**flightsdb**), caratterizzato da due tabelle, una per la gestione della lista di aeroporti di interesse per gli utenti e una per la persistenza dei dati riguardanti i voli aerei.
 - Fornisce funzionalità di elaborazione dei dati, come il calcolo di statistiche aggregate.
- **Comunicazione tra i servizi**

La comunicazione tra Data Collector e User Manager avviene tramite un canale **gRPC**, utilizzato per verificare l'esistenza degli utenti prima di soddisfare le richieste sui dati di volo. Questa scelta consente una comunicazione efficiente e tipizzata, riducendo il *coupling* tra i servizi e migliorando l'affidabilità complessiva del sistema.

3. Componenti e flussi operativi

Questo capitolo descrive nel dettaglio i componenti software del sistema e il loro funzionamento, con particolare attenzione alle API esposte dai servizi e ai flussi di interazione tra di essi.

3.1. Servizio User Manager

Il servizio **User Manager** è responsabile della gestione degli utenti del sistema e fornisce un insieme di API REST per la registrazione e la cancellazione degli utenti.

Le operazioni di registrazione sono implementate secondo una politica *at-most-once*, al fine di evitare la creazione di duplicati in presenza di richieste ripetute; in particolare, il servizio assume che a ogni variazione dei dati della richiesta da parte del frontend/client corrisponda un diverso `request_id`, così da distinguere correttamente richieste semanticamente differenti.

Le credenziali degli utenti vengono gestite in modo sicuro: le password non sono mai memorizzate in chiaro, ma vengono trattate mediante **hashing** e **salting**, in linea con le buone pratiche di base nella gestione delle credenziali.

API REST esposte

- **POST /user**

Permette la creazione di un nuovo utente.

La richiesta deve includere un identificatore univoco (`request_id`) nell'header, utilizzato per garantire l'idempotenza dell'operazione.

Prima di procedere alla creazione dell'utente, il servizio verifica se la richiesta è già stata elaborata consultando una cache Redis; in caso affermativo, viene restituita la risposta precedentemente memorizzata.

In caso contrario, l'utente viene creato nel database e il risultato dell'operazione viene salvato in cache.

- **DELETE /user/{user_id}**

Permette la cancellazione di un utente esistente.

Anche in questo caso viene applicata la politica *at-most-once* tramite l'utilizzo della cache, evitando l'elaborazione multipla della stessa richiesta.

3.2 Servizio Data Collector

Il **Data Collector** è responsabile della gestione degli aeroporti di interesse degli utenti e del recupero e dell'elaborazione dei dati sui voli.

Il servizio espone diverse API REST e comunica con lo User Manager tramite gRPC per verificare l'esistenza degli utenti.

Verifica dell'utente tramite gRPC

Prima di elaborare qualsiasi richiesta con email, il Data Collector verifica che l'email fornita corrisponda a un utente registrato, interrogando lo User Manager tramite un servizio gRPC dedicato.

Questo meccanismo consente di mantenere un basso livello di *coupling* tra i servizi, demandando allo User Manager la gestione della logica relativa agli utenti.

API REST esposte

- **POST /user-interests**
Associa un nuovo aeroporto a un utente.
- **DELETE /user-interests**
Rimuove un aeroporto dalla lista degli interessi di un utente.
- **GET /user-interests**
Restituisce la lista degli aeroporti associati a un utente.
- **GET /user-interests/lastflights/{airport_code}**
Restituisce le informazioni relative all'ultimo volo in partenza e in arrivo per un determinato aeroporto.
- **GET /user-interests/traffic-average**
Calcola e restituisce la media del numero di voli in partenza e in arrivo negli ultimi N giorni per un aeroporto specificato.

Tutte le informazioni recuperate o calcolate vengono memorizzate o lette dal database **flightsdb** dedicato ai dati di volo.

Il **Data Collector** integra un job schedulato che, a intervalli regolari, interroga le API di OpenSky Network per recuperare i dati sui voli relativi agli aeroporti di interesse degli utenti.

Questo processo avviene in modo automatico e indipendente dalle richieste dell'utente, consentendo di mantenere aggiornato il database **flightsdb** e di ridurre il numero di chiamate alle API esterne durante le operazioni di consultazione.

L'intervallo di esecuzione del job è **parametrizzato tramite variabile d'ambiente** e può essere configurato a piacere; nel sistema proposto è impostato, di default, a **12 ore**.

L'avvio dei servizi è gestito in modo **sincronizzato** tramite Docker Compose, sfruttando meccanismi di *dependency management* e *health check* per garantire che ciascun componente venga avviato solo quando le dipendenze necessarie sono disponibili.

In particolare, il servizio **User Manager** viene avviato solo dopo che il database **userdb** e il servizio **Redis** risultano operativi (*healthy*). Analogamente, il **Data Collector** viene avviato solo dopo che il database **flightsdb** è healthy e che il servizio User Manager sia stato avviato.

Questa configurazione consente di evitare problemi legati all'ordine di avvio dei container e contribuisce a rendere il sistema più stabile e prevedibile nella fase di inizializzazione.

Per ulteriori dettagli sugli endpoint, inclusi i parametri richiesti, le tipologie di dati e le possibili risposte, è possibile consultare la documentazione Swagger del progetto disponibile nella cartella **/docs**. Questa documentazione offre una panoramica completa delle API, facilitando la comprensione e l'utilizzo dei servizi esposti.

4. Tecnologie adottate e scelte progettuali

Questo capitolo descrive le principali tecnologie utilizzate nel progetto e ne motiva brevemente l'adozione. Le scelte sono state guidate dalla necessità di realizzare un sistema modulare, manutenibile e coerente con un'architettura a microservizi, in un contesto universitario.

Linguaggio di programmazione

Il progetto è stato sviluppato in **Python 3.12**, scelto per la sua semplicità, produttività e ampia disponibilità di librerie per lo sviluppo di servizi backend e sistemi distribuiti.

API REST

Le API REST sono implementate tramite **Flask**, framework web leggero che consente di definire endpoint in modo chiaro e flessibile. Flask risulta particolarmente adatto a un'architettura a microservizi, in cui ogni servizio espone funzionalità ben definite.

Comunicazione tra microservizi

La comunicazione interna tra i servizi avviene tramite **gRPC**, scelto per l'efficienza del protocollo binario e per la definizione esplicita delle interfacce tramite file .proto, riducendo l'overhead rispetto alle API REST.

Job schedulati

Le operazioni periodiche, come l'aggiornamento dei dati sui voli, sono gestite tramite **APScheduler**. Si tratta di una libreria Python ampiamente utilizzata e consolidata, particolarmente diffusa per la gestione di task pianificati in applicazioni backend.

Persistenza e caching

I dati relativi a utenti e voli sono memorizzati in **PostgreSQL**, database relazionale affidabile e transazionale. L'accesso avviene tramite il driver **psycopg**.

Per il caching e la gestione dell'idempotenza delle richieste è utilizzato **Redis**, migliorando le prestazioni e la robustezza del sistema.

Sicurezza

Le password degli utenti sono gestite in modo sicuro tramite **bcrypt**, utilizzato per l'hashing e salting, evitando la memorizzazione di credenziali in chiaro.

Containerizzazione e strumenti di supporto

Il sistema è containerizzato con **Docker** e orchestrato tramite **Docker Compose**, facilitando la gestione dei servizi e la riproducibilità dell'ambiente di esecuzione.

Il testing delle API è stato effettuato tramite **Postman**. Inoltre, sono stati implementati un **layer di logging** e un **layer di error handling** per ogni servizio, per garantire tracciabilità, coerenza nella gestione degli errori e maggiore manutenibilità del codice.

5. Conclusioni e possibili miglioramenti

Il progetto ha rispettato pienamente le richieste della consegna, realizzando un sistema dockerizzato a microservizi in grado di gestire utenti, recuperare e memorizzare dati sui voli tramite OpenSky Network, e offrire operazioni di registrazione e cancellazione utenti con politica "at-most-once".

Le scelte progettuali adottate hanno permesso di:

- Gestire la registrazione e cancellazione utenti tramite il servizio User Manager, con database PostgreSQL.
- Recuperare e aggiornare i dati sui voli con il Data Collector, includendo funzioni per l'ultimo volo e la media dei voli degli ultimi X giorni.
- Comunicare in modo efficiente tra servizi tramite gRPC.
- Automatizzare operazioni periodiche tramite job schedulati.
- Utilizzare cache in memoria per rispettare i requisiti di idempotenza e migliorare le prestazioni.

- Containerizzare i servizi e orchestrare l'intero stack per facilitare il deploy e la riproducibilità dell'ambiente.

Considerazioni sui miglioramenti

Alcune scelte progettuali sono state semplificate per motivi di tempistiche e di focus del progetto:

- L'endpoint **GET /user-interests** che accetta l'email come parametro può risultare meno sicuro, poiché le informazioni potrebbero essere loggate; in contesti reali è preferibile evitare di inviare dati sensibili nei parametri. Per la stessa ragione, la rimozione degli utenti viene gestita tramite un identificatore numerico univoco passato nell'URI. Tuttavia, l'utilizzo di parametri di tipo stringa è generalmente più sicuro rispetto all'invio di dati sensibili direttamente nell'URI, poiché quest'ultima viene quasi sempre loggata mentre i parametri a volte no.
- I controlli regex su email e password sono stati saltati, ma in produzione dovrebbero essere implementati per validazione e sicurezza.
- Le credenziali di OpenSky sono state incluse nel repository per semplicità e testing del progetto universitario; in un progetto reale andrebbero gestite tramite secret manager.
- Il sistema è limitato dalle restrizioni delle API di OpenSky (account gratuito), che impongono limiti di chiamate e accesso ai dati.

In generale, il progetto rappresenta un prototipo completo, con margini di miglioramento possibili in ambito sicurezza e validazione dati, senza compromettere il funzionamento delle funzionalità principali.